CSE557 · BIG DATA ANALYTICS · 5TH SEM

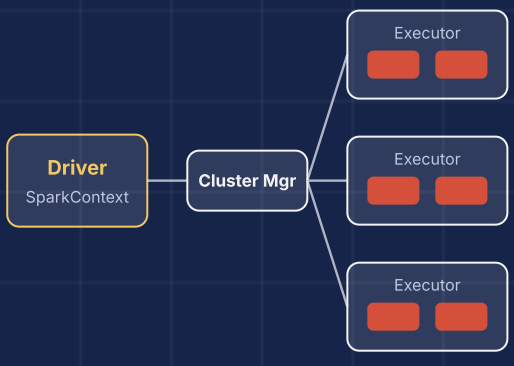
Unit 3
Spark Overview

CIE-1 portion only — Apache Spark architecture & installation, explained simply, with diagrams, tips and solved questions.

1 **Apache Spark Architecture**
What is Spark · Components · Terminology · Driver/Executor/Cluster manager · Job execution · Run-time flow

2 **Installation & Running Spark**
Installing Spark · First Spark program · Cluster manager types · Running Spark examples

Not included (comes after CIE-1): RDDs, transformations & actions, pair RDDs, partitioning, accumulators, broadcast variables.



Textbook: Muhammad Asif Abbasi, *Learning Apache Spark 2* (Packt) CIE-1 Edition

How to use this guide

Unit 3 is about **Apache Spark** — the faster successor to Hadoop MapReduce. For CIE-1 you need only the first block of the syllabus: **what Spark is, how its architecture works, and how to install and run it**. You already know Hadoop from Unit 2, so this guide keeps comparing Spark with it.

What the coloured boxes mean

🔑 **KEY POINT**
Must-know fact. If you remember nothing else, remember these.

🎯 **EXAM TIP**
How this topic is usually asked and how to score full marks.

💡 **TIP**
A shortcut to understand or study the idea faster.

🧠 **MEMORY TRICK**
Mnemonics to recall lists and step orders quickly.

⚠️ **COMMON MISTAKE**
Things students mix up — avoid losing marks here.

📌 **EXAMPLE**
A real-life example that makes the concept click.

Also: yellow highlighted text = exam keywords. Sections marked FROM TEXTBOOK are topics listed on your slide agenda but not explained on the slides — they are written from the prescribed textbook.

Suggested study plan (about 3 hours total)

Session 1
Chapter 1 — What is Spark, components, terminology
≈ 1 hr

Session 2
Chapter 1 — Architecture, job execution, run-time flow
≈ 1 hr

Session 3
Chapter 2 — Install, first program, cluster managers + cheat sheet
≈ 1 hr

🎯 **ABOUT CIE-1**

CIE-1 covers CO1–CO3: Unit 1, Unit 2 and **the first part of Unit 3**. CO3 is "Apply Apache Spark programming concepts ... and **cluster management**". The questions most likely from this part are **"Explain the Spark architecture with a neat diagram"**, **"Explain the components of Spark"**, **"Explain the run-time flow of a Spark application"** and **"Explain the cluster manager types"**.

Contents

Chapter 1 · Apache Spark Architecture 4

1.1 What is Apache Spark? 4

1.2 Spark components (the Spark stack) 5

1.3 Apache Spark terminology 6

1.4 Internal working — master/slave 7

1.5 Spark architecture overview 8

1.6 Internals of job execution (DAG & Task Scheduler) 10

1.7 Run-time architecture of a Spark application 10

1.8 Spark vs Hadoop MapReduce 11

Chapter 2 · Installation & Running Spark 13

2.1 Installing Apache Spark (Ubuntu) 13

2.2 Writing your first Spark program 15

2.3 Apache Spark cluster manager types 17

2.4 Running Spark examples 19

Final Revision 21

Cheat sheet 21

All memory tricks in one place 22

Important exam questions 22

CHAPTER 1

Apache Spark Architecture

What you'll learn

1. What Spark is and why it's fast
2. The 6 Spark components
3. SparkContext, Spark Shell, application
4. Job, stage and task
5. Master/slave: driver & executors
6. Driver, executor, cluster manager, worker
7. DAG Scheduler & Task Scheduler
8. Run-time flow, step by step

⏱ ≈ 2 hours | Slides: Spark Architecture (pp. 1–16) | Abbasi Ch. 1

1.1 What is Apache Spark?

IN SIMPLE WORDS

Spark is a tool for **processing large datasets quickly**. Like Hadoop MapReduce, it spreads work over many computers — but it keeps data **in memory (RAM)** between steps instead of writing to disk every time, so it is **much faster**. It works on **one computer or many computers**.

DEFINITION

Apache Spark is an open-source cluster computing framework for data analytics, designed for **quick computation** using a simple cluster technology. It supports in-memory cluster computing and **parallel processing**, and promises to be faster than Hadoop.

Key features (from your slides)

- Used for **data processing** — a tool for processing large datasets quickly.
- Works on **one computer or many computers** (a cluster).
- Supports **in-memory** cluster computing and **parallel processing**.
- Supports various **high-level tools** for data analysis (SQL, streaming, ML, graphs).
- Provides APIs for Scala, Java and Python (plus SQL and R).

A little history FROM TEXTBOOK

- Started in **2009** at UC Berkeley's **AMPLab** by **Matei Zaharia**.
- Open-sourced in 2010; became an **Apache top-level project in 2014**.
- Written mainly in **Scala**; runs on the Java Virtual Machine.
- Claimed to run programs up to 100× faster than Hadoop MapReduce in memory, and about **10× faster on disk**.

- In your course you will use Python (PySpark) on one Ubuntu machine.

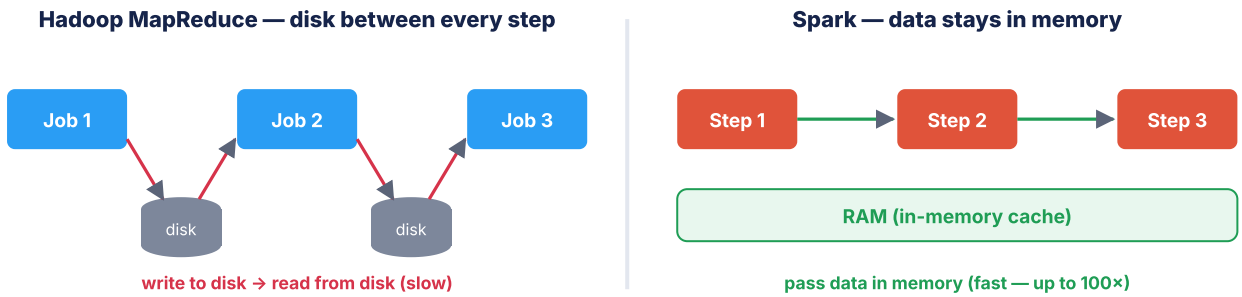


Figure 1.1 — Why Spark is faster than Hadoop MapReduce for multi-step work

> 🔑 **KEY POINT**
>
> Spark's big idea is in-memory computing. That makes it great for **iterative** work (machine learning, which loops over the same data many times) and **interactive** work (asking many quick questions of the same data).

1.2 Spark components (the Spark stack)

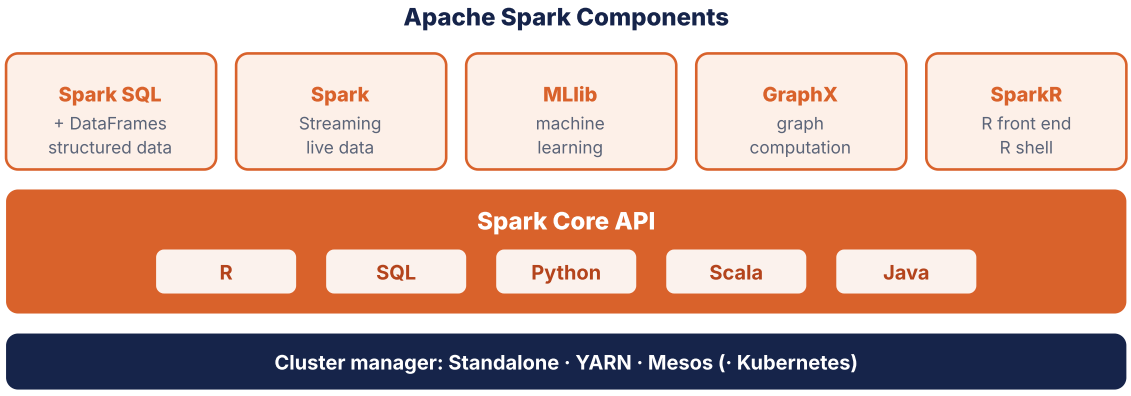


Figure 1.2 — The Spark stack: libraries on top of Spark Core, running on a cluster manager

⚙️ Spark Core

The foundation everything else runs on — provides the **execution platform** for every Spark application (task scheduling, memory management, fault recovery, talking to storage). Language APIs: R, SQL, Python, Scala, Java.

🗃 Spark SQL

Lets you run **SQL and HiveQL queries** against **structured and semi-structured data**, often meaningfully **faster** than the same queries on older systems. Comes with **DataFrames**.

🌊 Spark Streaming

Processes live data by breaking it into **small micro-batches** that run on top of Spark Core.

e.g., live tweets, sensor feeds, server logs.

🤖 MLlib

Spark's **machine learning library**, built to use **in-memory processing** for **iterative algorithms** that would otherwise be slow. (Unit 4 topic.)

🕸 GraphX

A **graph computation engine** for processing **graph-structured data** at scale.

e.g., social networks, page rank.

📊 SparkR

A lightweight **front end for R users** to analyse large datasets and run jobs interactively from the **R shell** — R's usability + Spark's scalability.

🧠 **MEMORY TRICK — 6 COMPONENTS**

"**C**ool **S**tudents **S**tream **M**ovies, **G**raphs & **R**eels" → **C**ore · **S**QL · **S**treaming · **M**Llib · **G**raphX · Spark**R**. Core is the floor; the other five sit on top.

1.3 Apache Spark terminology

SPARKCONTEXT

The entry point of a Spark application. It **connects to the Spark execution environment** and is used to **create RDDs, accumulators and broadcast variables**, access Spark services, and **run jobs**.

Responsibilities of SparkContext

1. Checking the **current status** of a Spark application
2. **Cancelling** a job or a stage
3. Running a job **synchronously or asynchronously**
4. Accessing or **unpersisting a cached RDD**
5. Programmable **dynamic allocation** of resources

💻 Spark Shell

A Spark application written in **Scala** that gives you a command-line environment with auto-completion. Useful for getting familiar with Spark's features **before writing a standalone application**. (`spark-shell` for Scala, `pyspark` for Python.)

📦 Spark Application

A self-contained computation that runs user-supplied code to produce a result. It can have processes running on its behalf **even when it is not actively running a job** (its executors stay alive).

🧠 **MEMORY TRICK**

"**S C R A D**" → **S**tatus · **C**ancel · **R**un · **A**ccess cache · **D**ynamic allocation — "**S**park **C**an **R**un **A**ny **D**ata".

💡 **SPARKSESSION** FROM TEXTBOOK

From **Spark 2.0**, `SparkSession` is the single, unified entry point. It wraps the SparkContext (plus SQLContext and HiveContext). Get the context with `spark.sparkContext`.

Task, Job and Stage

🔹 Task

The smallest unit of work, sent to a **single executor**. **Each partition** of an RDD gets its own task.

🔷 Job

A parallel computation made up of multiple tasks, created **in response to an action** (e.g., `count()`, `collect()`).

🧱 Stage

A set of tasks within a job that depend on each other. A job is split into stages because **not every computation can happen in one pass** — some steps must wait for others (e.g., a shuffle) to finish first.

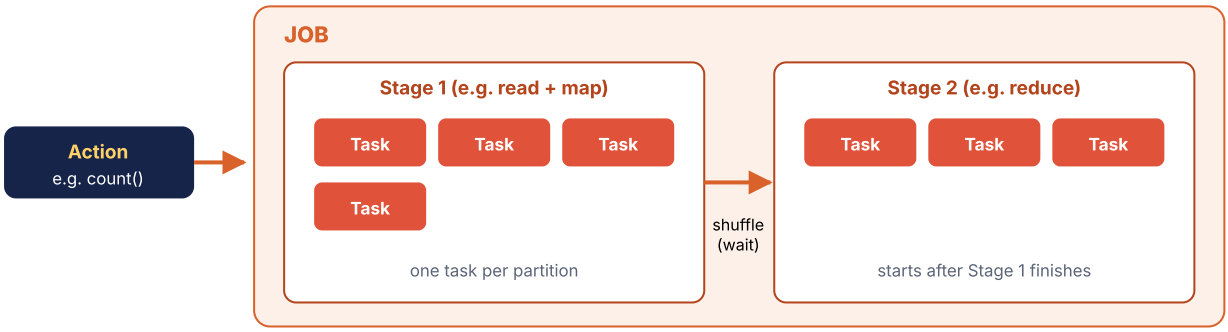


Figure 1.3 — An action creates a job; the job is split into stages; each stage runs many tasks

🧠 **MEMORY TRICK — SIZE ORDER**

Application ⊃ Job ⊃ Stage ⊃ Task — "**A**ll **J**obs **S**plit into **T**asks". An application can run many jobs; each action = one job.

⚠️ **DON'T CONFUSE WITH HADOOP (UNIT 2)**

In Hadoop, a **job** = one map phase + one reduce phase. In Spark, a **job** is triggered by an **action** and can have **many stages** — so one Spark job can do the work of several chained MapReduce jobs.

1.4 Internal working of Spark — master/slave

🗣️ **IN SIMPLE WORDS**

Think of a **restaurant**. The **driver** is the head chef who reads the order (your program) and plans the work. The **executors** are the cooks who actually chop and cook (process data). The **cluster manager** is the restaurant manager who decides how many cooks and stoves each head chef gets.

- Similar to Hadoop, Spark also uses a master–slave architecture: **one master and many slaves**.
- The driver is the central coordinator.
- The driver and each executor **run in their own Java process**. Drivers handle a **large number of executors**; each executor works as a **separate Java process**.
- A Spark application is a collaboration of a driver and its executors.
- Spark has its **own cluster manager**, the standalone cluster manager. It can also work with open-source cluster managers like YARN and Mesos.

1.5 Apache Spark architecture overview

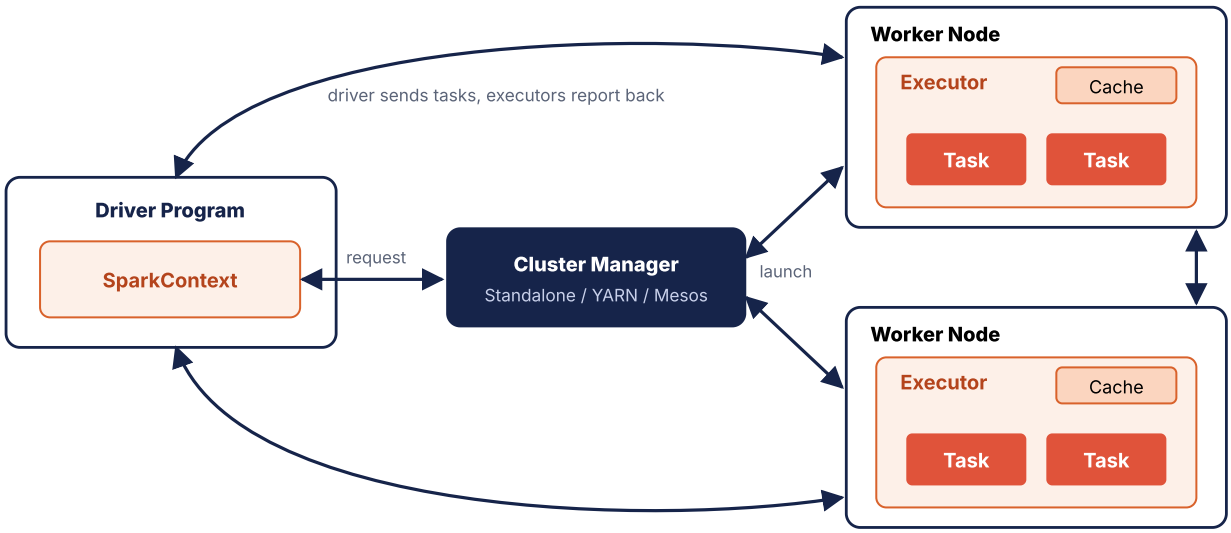


Figure 1.4 — Apache Spark architecture overview (driver program, cluster manager, worker nodes)

> 🎯 **EXAM TIP — THIS IS THE DIAGRAM**
> "Explain the Spark architecture with a neat diagram" is the most likely question from this part. Draw **Driver Program (SparkContext) ↔ Cluster Manager ↔ Worker Nodes (Executor + Cache + Tasks)**, plus the direct arrows from driver to executors. Then explain each box using the table and cards below.

The three main roles

Driver	Executor	Cluster Manager
• **Job scheduling** & **negotiation with the cluster manager** • **Stores RDD information** (metadata — how each RDD is built)	• **Performs all data processing** • **Stores computation results in memory** (cache)	• **Acquiring resources** from the Spark cluster • e.g., **YARN, Mesos** (or Standalone)

Each part in detail

🧠 Driver Program

- A process that runs the main() function of the application and creates the SparkContext object.
- The purpose of SparkContext is to **coordinate applications**, running as independent sets of processes on a cluster.
- Can run in **client mode or cluster mode** on different types of cluster managers.
- Tasks it performs: (1) it requires user code or methods to be **cleaned** (prepared to ship); (2) then it works over the application code to **process it**. The application code can be given as **JAR or Python files** passed to the SparkContext.

🧭 Cluster Manager

- Its role is to allocate resources across applications.
- Spark is capable of running on a **large number of clusters**.
- Various types: Hadoop YARN, Apache Mesos and Standalone Scheduler.
- The **Standalone Scheduler** is a standalone Spark cluster manager that makes it easy to install Spark on an **empty set of machines**.

🖥 Worker Node

- The worker node is a slave node.
- Its role is to **run the application code** in the cluster.

👷 Executor

- A process launched for an application on a worker node.
- It **runs tasks** and keeps data in **memory or disk storage** across them.
- It **reads and writes data** to external sources.
- **Every application has its own executors.**

🔹 Task

A **unit of work that will be sent to one executor**.

🧠 MEMORY TRICK — RESTAURANT AGAIN

Driver = head chef (plans, creates SparkContext) · **Cluster Manager** = restaurant manager (gives out stoves) · **Worker node** = kitchen station · **Executor** = cook at that station (cooks + keeps ingredients ready = cache) · **Task** = one dish order.

⚠️ COMMON MISTAKE

The **cluster manager does not run your code** and does not schedule tasks — it only **gives resources** (launches executors). **Task scheduling is done by the driver**.

1.6 Internals of job execution in Spark

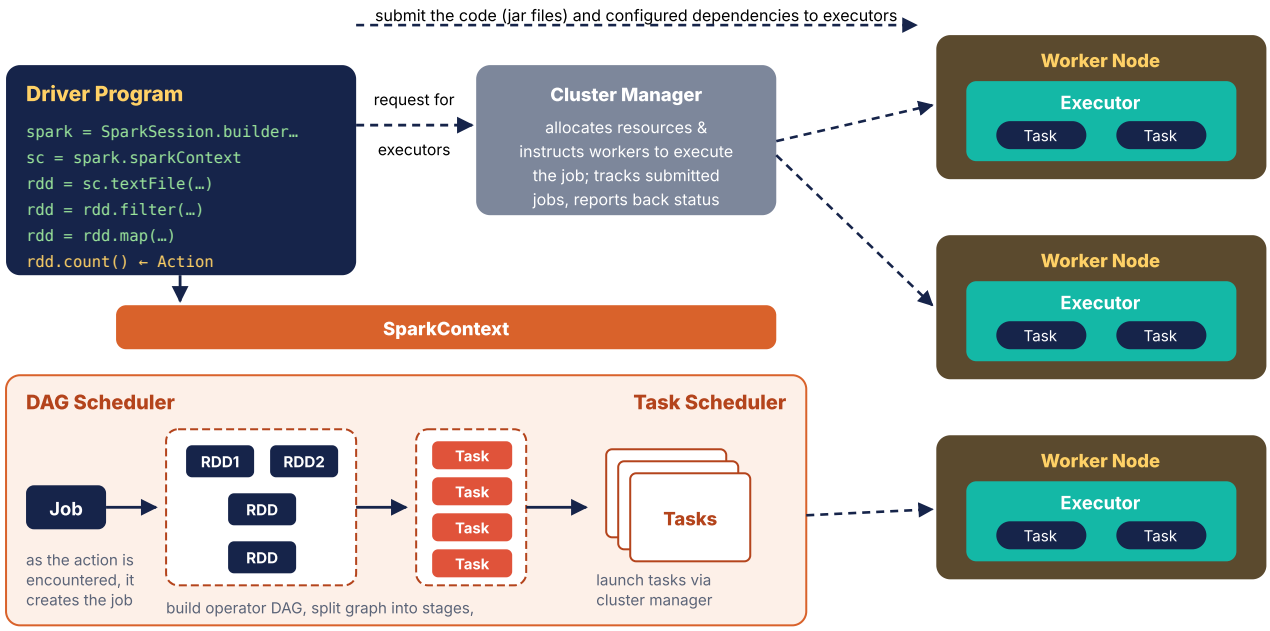


Figure 1.5 — Internals of job execution in Spark: Driver → SparkContext → DAG Scheduler → Task Scheduler → executors

1. The **driver program** creates a `SparkSession` / `SparkContext` and defines a chain of RDD operations (`textFile → filter → map`). Nothing runs yet.
2. When an **action** (e.g., `count()`) is encountered, a job is created.
3. The DAG Scheduler builds the **operator DAG** (graph of RDDs), **splits the graph into stages of tasks**, and submits each stage to the Task Scheduler as soon as it's ready.
4. The Task Scheduler **launches the tasks via the cluster manager** onto executors.
5. The **cluster manager** allocates resources and instructs workers to execute the job; it **tracks submitted jobs and reports back their status**.
6. The application code (**JAR files**) and configured dependencies are sent to the **executors** on worker nodes, which run the tasks.

> 🔑 **DAG = DIRECTED ACYCLIC GRAPH**
>
> A graph of steps with arrows that **only go forward** (no loops). Spark records your operations as a DAG first, **optimises** it, then runs it — this is why Spark can combine many steps into one stage and be faster than MapReduce.

1.7 Run-time architecture of a Spark application

> 🗣 **IN SIMPLE WORDS**
>
> From start to end: the driver **asks for workers**, the workers **report for duty**, the driver **turns your code into a plan of stages and tasks**, sends the tasks out, **watches** them run, and when you call `stop()` it **sends everyone home** and gives the resources back.



Figure 1.6 — Run-time flow of a Spark application (10 steps)

1. The **driver talks to the cluster manager** and **negotiates for resources**.
2. The **cluster manager launches executors** on behalf of the driver.
3. The **executors register themselves** with the driver.
4. When a job enters, the driver **converts the code into a logical directed acyclic graph (DAG)**.
5. The DAG is **optimised**.
6. It is turned into a **physical execution plan** with a **set of stages**.
7. Small execution units known as tasks are created and **sent to the cluster**.
8. **Executors execute all the tasks** assigned by the driver. The **driver monitors** the executors that run.
9. The driver **schedules future tasks** by **tracking the location of cached data** (based on data placement) — so tasks run where their data already is.
10. When the program calls the `stop()` method of SparkContext, it **terminates all executors** and **releases the resources** from the cluster manager.

> 🧠 **MEMORY TRICK — 10 STEPS**
>
> "**N L R — L O P T — E M S**": **N**egotiate · **L**aunch · **R**egister → **L**ogical DAG · **O**ptimise · **P**hysical plan · **T**asks → **E**xecute · **M**onitor · **S**top.
> Story: "**N**ew **L**abourers **R**eport; the boss **L**ists, **O**rganises, **P**lans, gives **T**asks; they **E**xecute, he **M**onitors, then **S**ays stop."

> 💡 **COMPARE WITH HADOOP YARN (UNIT 2)**
>
> The pattern is the same as YARN: **ask for resources → start workers → run tasks → monitor → release**. In Spark the **driver** plays the role that the **ApplicationMaster** plays in MapReduce on YARN. When Spark runs *on* YARN, the YARN ResourceManager is the cluster manager.

1.8 Spark vs Hadoop MapReduce

A favourite exam comparison — it connects Unit 2 and Unit 3.

Point	Hadoop MapReduce	Apache Spark
Processing	Disk-based — writes intermediate results to disk	**In-memory** — keeps intermediate data in RAM
Speed	Slower	Up to **100× faster in memory**, ~10× on disk
Programming model	Only map and reduce	Many operations (map, filter, join, groupBy...) on RDDs/DataFrames
Execution plan	Fixed: map → shuffle → reduce	**DAG** of stages, optimised before running
Iterative / interactive work	Poor (each iteration = new job + disk I/O)	Excellent (data cached in memory; interactive shell)
Real-time	Batch only	Batch + streaming (Spark Streaming)
Languages	Mainly Java	Scala, Java, Python, R, SQL
Libraries	Separate tools (Hive, Pig, Mahout...)	Built-in: SQL, Streaming, MLlib, GraphX, SparkR
Cluster manager	YARN	Standalone, YARN, Mesos, Kubernetes
Storage	HDFS	No storage of its own — reads from HDFS, local files, S3, HBase, Cassandra...
Master / workers	JobTracker/TaskTracker (MR1); RM/AM/NM (YARN)	Driver + executors (+ cluster manager)

⚠️ **COMMON MISTAKE**

Spark does **not replace HDFS** — Spark is a **processing engine**, not a storage system. A very common setup is **Spark on YARN reading from HDFS**.

✅ **Before you move on — can you...**

- ☐ Define Spark and explain why in-memory computing makes it faster than MapReduce?
- ☐ Draw the Spark stack and explain the 6 components?
- ☐ Define SparkContext (+ 5 responsibilities), Spark Shell and Spark Application?
- ☐ Explain task, job and stage with a diagram?
- ☐ Explain the master/slave internal working (driver, executors, Java processes)?
- ☐ Draw the architecture diagram and explain driver, cluster manager, worker node, executor and task?
- ☐ Explain the internals of job execution (DAG Scheduler, Task Scheduler)?
- ☐ List the 10 run-time steps in order?
- ☐ Compare Spark with Hadoop MapReduce?

CHAPTER 2

Installation & Running Spark

What you'll learn

1. Installing Spark on Ubuntu
2. Spark's folders & shells
3. Your first program in the shell
4. A standalone word-count app
5. spark-submit
6. Local, Standalone, YARN, Mesos, Kubernetes
7. Client vs cluster deploy mode
8. Running the bundled examples

⏱ ≈ 1 hour | Slide agenda topics 2–5 | Abbasi Ch. 1

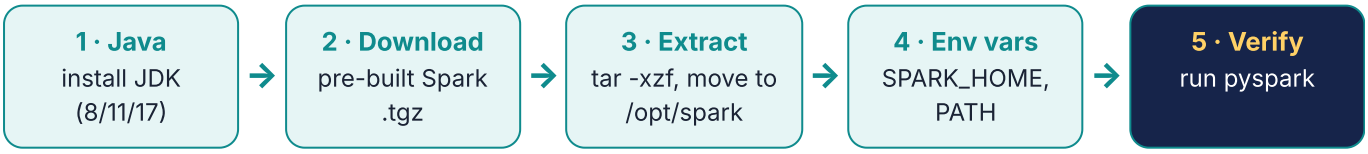
⚠️ **NOTE ABOUT THIS CHAPTER**

Your slide deck **lists** these four topics in its agenda (Installing Spark, first program, cluster manager types, running examples) and says **"We will use Python (PySpark) on one Ubuntu machine"**, but it doesn't explain them in detail. This chapter is written from the **prescribed textbook** and the official Spark docs. Version numbers are examples — use the version your lab uses.

2.1 Installing Apache Spark (Ubuntu) FROM TEXTBOOK

🗣 **IN SIMPLE WORDS**

Spark runs on **Java**, so install Java first. Then download Spark, unzip it, tell your terminal where it is (environment variables), and start the **pyspark** shell to check it works. That's it — no cluster needed; one laptop is enough to learn.



```
# 1. Prerequisite: Java (and Python 3 for PySpark)
sudo apt update
sudo apt install -y openjdk-11-jdk python3
java -version                                  # check Java is installed

# 2. Download a pre-built Spark package from spark.apache.org/downloads
wget https://archive.apache.org/dist/spark/spark-3.5.1/spark-3.5.1-bin-hadoop3.tgz

# 3. Extract it and move it to /opt/spark
tar -xzf spark-3.5.1-bin-hadoop3.tgz
sudo mv spark-3.5.1-bin-hadoop3 /opt/spark

# 4. Set environment variables (add these lines to ~/.bashrc)
export SPARK_HOME=/opt/spark
export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin
export PYSPARK_PYTHON=python3
source ~/.bashrc

# 5. Verify
spark-shell --version                          # prints the Spark version banner
pyspark                                        # opens the Python shell with 'spark' and 'sc' ready
```

💡 **EVEN QUICKER: PIP**

For Python-only use on one machine: `pip install pyspark` — this installs Spark inside Python. You still need Java installed.

What's inside the Spark folder

Folder	Contains
`bin/`	Commands you run: `spark-shell` (Scala), `pyspark` (Python), `sparkR` (R), `spark-sql`, `spark-submit` (submit apps), `run-example`
`sbin/`	Scripts to start/stop a **standalone cluster**: `start-master.sh`, `start-worker.sh` (called `start-slave.sh` in Spark 2), `start-all.sh`, `stop-all.sh`
`conf/`	Configuration templates: `spark-env.sh`, `spark-defaults.conf`, `log4j` settings, `workers` list
`examples/`	Ready-made example programs (SparkPi, WordCount…) in Scala, Java, Python and R
`jars/` · `python/` · `R/`	Spark's libraries for each language
`data/`	Small sample datasets used by the examples

The Spark shells

spark-shell
Interactive **Scala** shell (the "Spark Shell" from §1.3).

pyspark
Interactive **Python** shell — what your lab uses.

sparkR
Interactive **R** shell (SparkR).

spark-sql
Type **SQL** queries directly.

🔑 **INSIDE A SHELL**

When a shell starts, Spark **already creates the SparkContext (`sc`) and SparkSession (`spark`) for you**. While it runs, the Spark Web UI is at `http://localhost:4040` — it shows jobs, stages, tasks, storage and executors (great for seeing §1.3 and §1.6 in action).

2.2 Writing your first Spark program FROM TEXTBOOK

A. Interactively in the shell

The textbook's first program loads Spark's own `README.md` as a text file and asks simple questions about it.

In `pyspark` (Python)

```
>>> lines = sc.textFile("README.md")   # an RDD
of lines
>>> lines.count()                  # action → job
125
>>> lines.first()                  # first line
'# Apache Spark'
>>> spark_lines = lines.filter(
...     lambda l: "Spark" in l)        #
transformation
>>> spark_lines.count()
20
```

In `spark-shell` (Scala)

```
scala> val lines = sc.textFile("README.md")
scala> lines.count()
res0: Long = 125
scala> lines.first()
res1: String = # Apache Spark
scala> val sparkLines =
     |   lines.filter(l => l.contains("Spark"))
scala> sparkLines.count()
res2: Long = 20
```

(The exact numbers depend on the Spark version's README.)

💡 **CONNECT IT TO THE ARCHITECTURE**

`textFile()` and `filter()` only **record steps in the DAG** — nothing runs. `count()` and `first()` are **actions**: each one creates a **job**, which the DAG Scheduler splits into **stages** and **tasks** (one per partition) that run on executors (§1.6). Transformations vs actions are studied in detail after CIE-1.

B. A standalone application — word count

A real application creates its own SparkSession (the shell did it for you before). Save this as `wordcount.py` :

```python
from pyspark.sql import SparkSession

# 1. Create the entry point (driver creates SparkSession / SparkContext)
spark = (SparkSession.builder
         .appName("WordCount")
         .master("local[*]")        # run locally using all CPU cores
         .getOrCreate())
sc = spark.sparkContext

# 2. Load data → 3. transform → 4. action
lines  = sc.textFile("input.txt")
words  = lines.flatMap(lambda line: line.split())     # split lines into words
pairs  = words.map(lambda w: (w, 1))                  # (word, 1) — like a mapper
counts = pairs.reduceByKey(lambda a, b: a + b)        # add up — like a reducer

for word, n in counts.collect():                        # action → runs the job
    print(word, n)

# 5. Stop — terminates executors, releases resources (run-time step 10)
spark.stop()
```

Run it with **spark-submit**:

```bash
spark-submit wordcount.py
# or choose where it runs:
spark-submit --master local[4] wordcount.py
```

> 📌 **SAME IDEA AS UNIT 2**
>
> This is the classic **word count**: `map` emits `(word, 1)` like a **Mapper**, `reduceByKey` sums them like a **Reducer** — but in about **6 lines** instead of three Java classes, and the data stays in memory between steps.

spark-submit — the main options

Option	Meaning
`--master`	Which cluster manager / where to run (see §2.3): `local[*]`, `spark://host:7077`, `yarn` …
`--deploy-mode`	`client` (driver on your machine) or `cluster` (driver inside the cluster)
`--class`	Main class of a Java/Scala application (not needed for Python)
`--name`	Application name shown in the Web UI
`--executor-memory`, `--executor-cores`, `--num-executors`	Resources to ask the cluster manager for (run-time step 1)
app file + arguments	The `.py` file or `.jar`, followed by its own arguments

2.3 Apache Spark cluster manager types

From the slides: the cluster manager's role is to **allocate resources across applications**. Spark has its **own standalone cluster manager**, and can also work with **Hadoop YARN** and **Apache Mesos**.

FROM TEXTBOOK Spark 2.3+ also supports **Kubernetes**, and **local mode** runs everything on one machine.

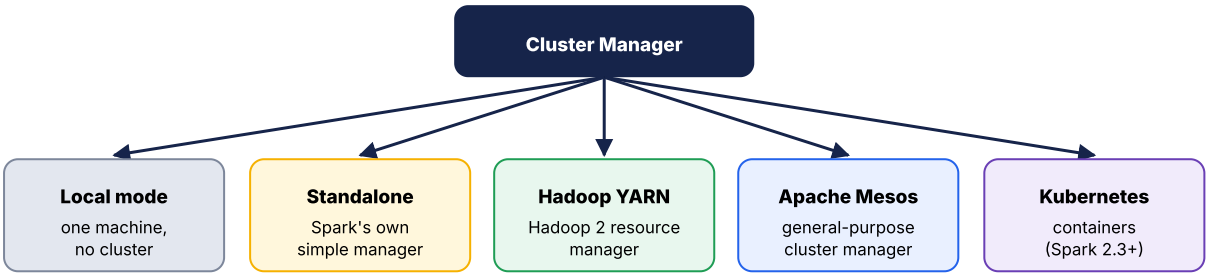


Figure 2.1 — Cluster manager types for Spark (slides list Standalone, YARN and Mesos)

🖥 Local mode

Driver and executor run in **one JVM on one machine** — no real cluster. Used for **learning, development and testing** (your Ubuntu lab setup). Master URL: `local` (1 thread), `local[4]` (4 threads), `local[*]` (all cores).

⭐ Standalone cluster manager

Spark's own simple cluster manager, shipped with Spark. Makes it easy to set up a cluster on an **empty set of machines**. One **master** + several **workers**. Start with `sbin/start-master.sh` then `sbin/start-worker.sh spark://<master>:7077`. Master UI on port **8080**. Master URL: `spark://HOST:7077`.

🐘 Hadoop YARN

The **resource manager of Hadoop 2** (Unit 2!). Lets Spark run on an **existing Hadoop cluster**, sharing resources with MapReduce and reading data from **HDFS**. The YARN ResourceManager acts as the cluster manager; executors run in YARN **containers**. Master URL: `yarn`.

🏛 Apache Mesos

A **general-purpose cluster manager** that can run Spark alongside other frameworks (and also Hadoop MapReduce). Supports fine-grained sharing of resources. Master URL: `mesos://HOST:5050`. (Deprecated in newer Spark versions.)

☸ Kubernetes FROM TEXTBOOK / DOCS

Runs the driver and executors as **pods (containers)** on a Kubernetes cluster. Master URL: `k8s://https://HOST:PORT`.

Master URLs at a glance

--master value	Runs on
`local`	Locally with **one** worker thread (no parallelism)
`local[K]`	Locally with **K** worker threads (e.g., `local[4]`)
`local[*]`	Locally with as many threads as **CPU cores**
`spark://HOST:7077`	A Spark **Standalone** cluster master (default port 7077)
`yarn`	A **YARN** cluster (location read from `HADOOP_CONF_DIR`)
`mesos://HOST:5050`	A **Mesos** cluster
`k8s://https://HOST:PORT`	A **Kubernetes** cluster

Deploy modes: client vs cluster

The slides say the driver "**can run in client or cluster mode**". The difference is **where the driver runs**:

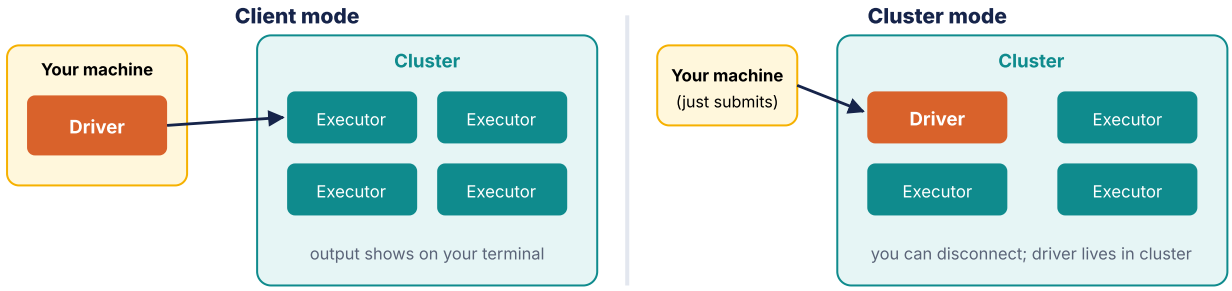


Figure 2.2 — Client vs cluster deploy mode

	Client mode	Cluster mode
Where the driver runs	On the machine that **submitted** the job	**Inside the cluster** (on YARN: inside the ApplicationMaster)
Good for	Interactive work, shells, debugging	Production jobs
If you close the terminal	The application stops	The application keeps running
Shells (pyspark, spark-shell)	✅ Always client mode	❌ Not possible

🧠 **MEMORY TRICK — CLUSTER MANAGERS**

"**L S Y M K**" → "**L**et **S**park **Y**ell **M**ore **K**indly" — **L**ocal · **S**tandalone · **Y**ARN · **M**esos · **K**ubernetes. Slides' three: **S Y M** ("Spark's Your Manager").

🎯 EXAM TIP

"Explain the cluster manager types in Spark" → draw Fig 2.1, explain **Standalone, YARN and Mesos** (the three on your slides) with their master URLs, add one line each on local mode and Kubernetes, then add the **client vs cluster** deploy mode table.

2.4 Running Spark examples

FROM TEXTBOOK

Spark ships with ready-made examples in the `examples/` folder. Running them is the easiest way to check your installation and see a cluster at work.

SparkPi — estimating π

```
# Scala/Java examples: use run-example (the argument = number of partitions/slices)
./bin/run-example SparkPi 10
…  Pi is roughly 3.1418

# Python version of the same example: use spark-submit
./bin/spark-submit examples/src/main/python/pi.py 10
Pi is roughly 3.140960
```

📌 HOW SPARKPI WORKS (SIMPLE IDEA)

Throw lots of random "darts" at a 1×1 square. The fraction that land inside the quarter-circle ≈ π/4, so π ≈ 4 × (inside ÷ total). Spark splits the darts into **10 partitions → 10 tasks** that run in parallel on executors, then adds up the counts. More partitions = more parallel tasks.

Word count example

```
./bin/spark-submit --master local[4] examples/src/main/python/wordcount.py README.md
```

Running an example on different cluster managers

```
# Local (all cores)
./bin/spark-submit --master local[*]            examples/src/main/python/pi.py 100
# Standalone cluster
./bin/spark-submit --master spark://master:7077 examples/src/main/python/pi.py 100
# YARN, driver inside the cluster
./bin/spark-submit --master yarn --deploy-mode cluster examples/src/main/python/pi.py 100
```

💡 WATCH IT RUN

While an example runs, open `http://localhost:4040`. You'll see the **job** created by the action, its **stages**, and the **10 tasks** — the terminology from §1.3 made visible.

✅ **Before you move on — can you…**

- ☐ List the 5 steps to install Spark on Ubuntu (Java, download, extract, env vars, verify)?
- ☐ Name the important folders (bin, sbin, conf, examples) and the four shells?
- ☐ Write the first program (textFile, count, first, filter) and a word-count app?
- ☐ Explain what spark-submit does and its main options?
- ☐ Explain local, Standalone, YARN, Mesos (and Kubernetes) with their master URLs?
- ☐ Explain client vs cluster deploy mode?
- ☐ Run SparkPi with run-example and with spark-submit?

LAST-NIGHT REVISION

Final Revision

Use these pages the night before the exam

1. Cheat sheet
2. All memory tricks in one place
3. Important 2-mark questions
4. Important 5/10-mark questions

⏱ ≈ 20 minutes

Then redraw the architecture diagram once

★ Cheat sheet

CHAPTER 1 · Spark architecture

- **Spark** = open-source cluster computing framework for data analytics; **in-memory** cluster computing + parallel processing; faster than Hadoop (up to 100× in memory). APIs: Scala, Java, Python, (SQL, R). Lab: PySpark on Ubuntu.
- **Components:** Spark Core (foundation/execution platform) · Spark SQL (SQL & HiveQL on structured/semi-structured data) · Spark Streaming (live data as micro-batches) · MLlib (ML, iterative, in-memory) · GraphX (graphs) · SparkR (R front end).
- **SparkContext** = entry point; connects to execution environment; creates RDDs, accumulators, broadcast variables; runs jobs. Responsibilities: status · cancel job/stage · run sync/async · access/unpersist cached RDD · dynamic allocation. (Spark 2: SparkSession wraps it.)
- **Spark Shell** = Scala command line with auto-completion. **Spark Application** = self-contained computation running user code.
- **Task** = smallest unit, one executor, one per partition · **Job** = parallel computation of many tasks, created by an **action** · **Stage** = set of dependent tasks; jobs split where steps must wait (shuffle).
- **Master/slave:** one driver (central coordinator) + many executors; each is its own Java process; app = driver + its executors. Own **standalone** cluster manager, or YARN / Mesos.
- **Driver:** runs main(), creates SparkContext, job scheduling, negotiates with cluster manager, stores RDD info; client or cluster mode. **Executor:** process on a worker node; runs tasks; keeps data in memory/disk; reads/writes external data; one set per app. **Cluster manager:** allocates resources (YARN, Mesos, Standalone). **Worker node:** slave node running app code. **Task:** unit of work for one executor.
- **Job execution:** action → job → **DAG Scheduler** (build DAG, split into stages) → **Task Scheduler** (launch tasks via cluster manager) → executors on worker nodes.
- **Run-time (10):** negotiate → launch executors → register → logical DAG → optimise → physical plan (stages) → tasks to cluster → execute (driver monitors) → schedule by cached-data location → `stop()` kills executors, releases resources.
- **vs MapReduce:** in-memory vs disk; DAG vs fixed map/reduce; iterative & streaming; many languages; built-in libraries; no own storage (uses HDFS etc.).

CHAPTER 2 · Installation & running

- **Install:** Java (JDK) → download pre-built .tgz → `tar -xzf` → `SPARK_HOME` + `PATH` → `pyspark` / `spark-shell --version`. Or `pip install pyspark`.
- **Folders:** bin (shells, spark-submit, run-example) · sbin (start-master/worker) · conf · examples · jars · data.
- **Shells:** spark-shell (Scala), pyspark (Python), sparkR, spark-sql. `sc` & `spark` ready-made. Web UI at **localhost:4040**.
- **First program:** `sc.textFile("README.md")` → `count()`, `first()`, `filter("Spark")`. Word count: flatMap → map (w,1) → reduceByKey → collect; `spark.stop()`.
- **spark-submit:** --master, --deploy-mode, --class, --name, executor memory/cores.
- **Cluster managers:** local (local, local[K], local[*]) · Standalone (spark://HOST:7077, UI 8080) · YARN (yarn) · Mesos (mesos://HOST:5050) · Kubernetes (k8s://...).
- **Deploy modes:** client (driver on your machine; shells) vs cluster (driver inside cluster; production).
- **Examples:** `./bin/run-example SparkPi 10` → "Pi is roughly 3.14..."; Python: `spark-submit examples/src/main/python/pi.py 10`.

🧠 All memory tricks in one place

To remember…	Trick	Expands to
6 components	"Cool Students Stream Movies, Graphs & Reels"	Core, SQL, Streaming, MLlib, GraphX, SparkR
SparkContext responsibilities	"Spark Can Run Any Data" (S C R A D)	Status, Cancel, Run sync/async, Access cached RDD, Dynamic allocation
Size order	"All Jobs Split into Tasks"	Application ⊃ Job ⊃ Stage ⊃ Task
Architecture roles	Restaurant	Driver = head chef, Cluster manager = manager, Worker = station, Executor = cook, Task = one dish
Run-time 10 steps	"New Labourers Report; boss Lists, Organises, Plans, gives Tasks; they Execute, he Monitors, then Says stop"	Negotiate, Launch, Register, Logical DAG, Optimise, Physical plan, Tasks, Execute, Monitor/schedule, Stop
Cluster managers	"Let Spark Yell More Kindly"	Local, Standalone, YARN, Mesos, Kubernetes
Slides' 3 managers	"Spark's Your Manager" (S Y M)	Standalone, YARN, Mesos

🎯 Important exam questions — all the best for CIE-1! ⚡

Short answers (2–3 marks)

1. What is Apache Spark? List its features. §1.1
2. Why is Spark faster than Hadoop MapReduce? §1.1, §1.8
3. What is SparkContext? List its responsibilities. §1.3
4. Define task, job and stage. §1.3
5. What is the Spark Shell? §1.3
6. What is an executor? §1.5
7. What is the role of the driver program? §1.5
8. What is a DAG? What does the DAG Scheduler do? §1.6
9. List the cluster managers supported by Spark. §2.3
10. Client vs cluster deploy mode. §2.3
11. What does `local[*]` mean? §2.3
12. How do you run the SparkPi example? §2.4

Long answers (5–10 marks)

1. Explain the Apache Spark architecture with a neat diagram. §1.4–1.5
2. Explain the components of Apache Spark (Spark stack). §1.2
3. Explain Spark terminology: SparkContext, Spark Shell, application, job, stage, task. §1.3
4. Explain the internals of job execution in Spark. §1.6
5. Explain the run-time architecture of a Spark application. §1.7
6. Compare Apache Spark with Hadoop MapReduce. §1.8
7. Explain the steps to install Apache Spark and write your first Spark program. §2.1–2.2
8. Explain the different cluster manager types in Spark. §2.3

🎯 THE 3 DIAGRAMS YOU MUST BE ABLE TO DRAW

1. **Spark architecture** — Driver (SparkContext) ↔ Cluster Manager ↔ Worker Nodes (Executor, Cache, Tasks) (Fig 1.4)
2. **Spark components stack** — SQL, Streaming, MLlib, GraphX, SparkR on Spark Core (Fig 1.2)
3. **Job execution / run-time flow** — Driver → DAG Scheduler → Task Scheduler → executors (Fig 1.5, 1.6)